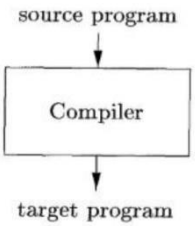
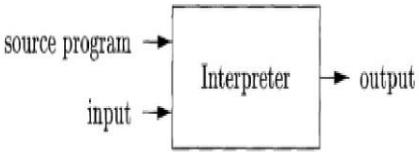
# Lec 1

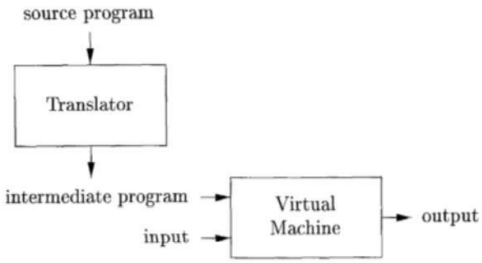
**Compiler**: is a program that can read a program in one language and translate it into an equivalent program in another language.



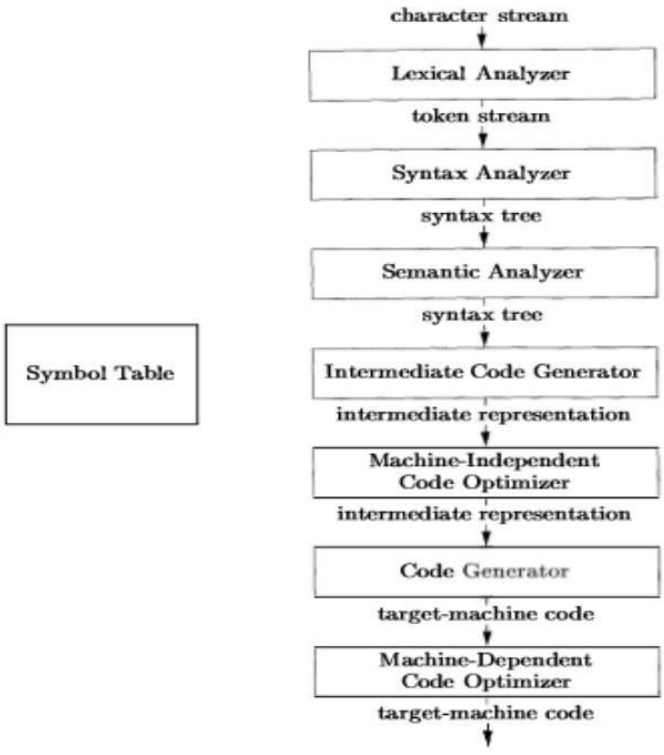

**interpreter:** Is directly execute the operations specified in the source program on inputs supplied by the user




**Java Lang processor:** Java language processor has a special feature that combine both of the compiler and interpreter.




## Phases of compiler



**Lexical analysis:** Break program down into its smallest meaningful symbols divided into a cascade of two processes:

- Scanning: deletion of comments & compaction characters
- Lexical analysis: produces the sequence of tokens as output

**Syntax Analysis:** second phase of the compiler, uses the first components to create a tree-like intermediate representation

**Semantic Analysis:** uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition

**Intermediate Code Generation:** compilers generate an intermediate representation which called sometimes (three-address instruction)

**Code optimization:** The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result.

**Code generator:** takes as input an intermediate representation of the source program and maps it into the target language.

# Lec 2,3

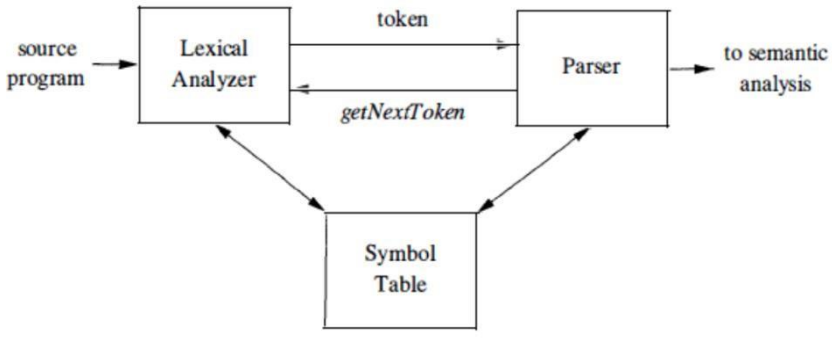
## The Role of the Lexical Analyzer:

1- Read the input characters of the source program

2- Group them into lexemes

3- Produce as output a sequence of tokens for each lexeme

## Interaction Between Lexical analyzer & Parser :




**Token**: is a pair consisting of a token name and an optional attribute value.

**Pattern:** is a description of the form that the lexemes of a token may take

**Lexeme:** is a sequence of characters in the source program that matches the pattern for a token

**Alphabet**: is any finite set of symbols.

**String:** is a finite sequence of symbols drawn from that alphabet.

**Language:** is any countable set of strings over some fixed alphabet.

**Terms for Parts of Strings:**

**1 - prefix:** of string s is any string obtained by removing zero or more symbols from the end of s.

**2- suffix:** of string s is any string obtained by removing zero or more symbols from the beginning of s.

**3- substring**: of s is obtained by deleting any prefix and any suffix from s.

**4- subsequence:** of s is any string formed by deleting zero or more not

**Operations on Languages**

- **Union:** is the familiar operation on sets.
- **The concatenation:** of languages is all strings formed by taking a string from the first language and a string from the second language

**•The (Kleene) closure:** is the set of strings you get by concatenating L

**Regular Expressions:** set is called regular if it can be generated from the elements of the alphabet using unio , concatenation and Kleene star operation.

**SDD (syntax Directed Definition) :** kind of specification generates from context free grammar in which each grammar production associated with set of production Rule

| PRODUCTION | SEMANTIC RULE |
|---|---|
| $E \rightarrow E_1 + T$ | $E.code = E_1.code \parallel T.code \parallel '+'$ |

**SDT (syntax Direct Translation)** : Argument Rules to the Grammar That Semantic analysis

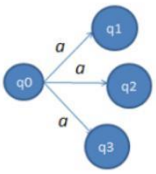
For example, consider an expression expr generated by the production :

$expr \rightarrow expr_1 + term$

# Lec 3,4

## Types of Finite Automata

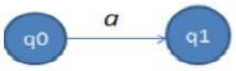
### • Nondeterministic Finite Automata (NFA):




- have no restrictions on the labels of their edges.

- the same I/P loads to more than one next state

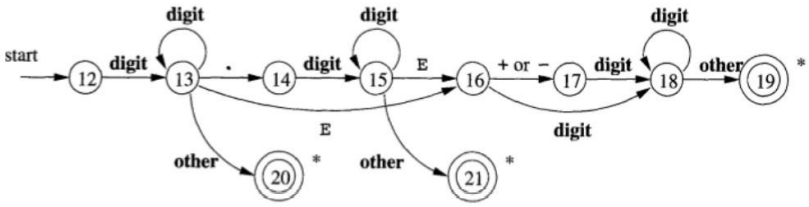
### • Deterministic Finite Automata (DFA):




- have for each state one edge

- the same I/P loads to only one next state

## Transition diagram for unsigned numbers




**Discuss (the input , output ,and method used to simulate a DFA**

**INPUT:** An input string x terminated by an end-of-file character eof, A DFA D with start state S0, accepting states F, and transition function move.

**OUTPUT:** Answer "yes" if D accepts x; "no" otherwise.

**METHOD:** Apply the algorithm in Fig. to the input string x. The function move(s, c) gives the state to which there is an edge from state s on input c. The function next Char returns the next character of the input string x.

# Lec 5,6

```
s = s0;
c = nextChar();
while ( c != eof ) {
        s = move(s, c);
        c = nextChar();
}
if ( s is in F ) return "yes";
else return "no";
```

**CFG (context-free grammar):** is used to specify the syntax of a language.

Component:

- set of nonterminal - start symbol - set of terminal - production rule

**leftmost derivations:** the leftmost nonterminal in each sentential is always chosen to be replaced. **rightmost derivations:** the rightmost nonterminal in each sentential is always chosen to be replaced.

**Ambiguous grammar:** is one that produces more than one leftmost derivation or more than one rightmost derivation for the same sentence.

**Top-down parsers:** Start at the root of the parse tree and grow toward leaves.

**Bottom-up parsing:** beginning at the leaves (the bottom) and working up towards the root (the top).

# Lec 7

**FOLLOW set(Algorithm):**

To build FOLLOW(X) for all non-terminal X:

1. Place $ in FOLLOW(X) // $ = EOF /End Marker

   iterate until no more terminals or ε can be added to any FOLLOW(X):

2. If A → αBβ then put {FIRST(β) - ε} in FOLLOW(B)
3. If A → αB then put FOLLOW(A) in FOLLOW(B)
4. If A → αBβ and ε ∈ FIRST(β) then put FOLLOW(A) in FOLLOW(B)

**Computing FIRST( Algorithm):**

To build FIRST(X) for all grammar symbols X:

1. if X is a terminal (token), FIRST(X) := { X }
2. if X ::= ε, then ε ∈ FIRST(X)
3. iterate until no more terminals or ε can be added to any FIRST(X):

   if X ::= $Y_1 Y_2 \dots Y_k$ then

   a ∈ FIRST(X) if a ∈ FIRST($Y_i$) and ε ∈ FIRST($Y_j$) for all $1 \le j < i$

   ε ∈ FIRST(X) if ε ∈ FIRST($Y_i$) for all $1 \le i \le k$

   end iterate

## Construction of a predictive parsing table

**INPUT:** grammar G

**OUTPUT:** Parsing table M

**METHOD:** For each production A → α of the grammar, do the following:

1. For each terminal α in FIRST(A), add A → α to M[A, α).

2 If € is in FIRST (α), then for each terminal b in FOLLOW(A), add A → α to M*A, b+. If € is in FIRST(α) and $ is in FOLLOW(A), add A → α to M[A, $] as well..

If, after performing the above, there is no production at all in M[A,α] then set M[A, α] to error (which we normally represent by an empty entry in the table).

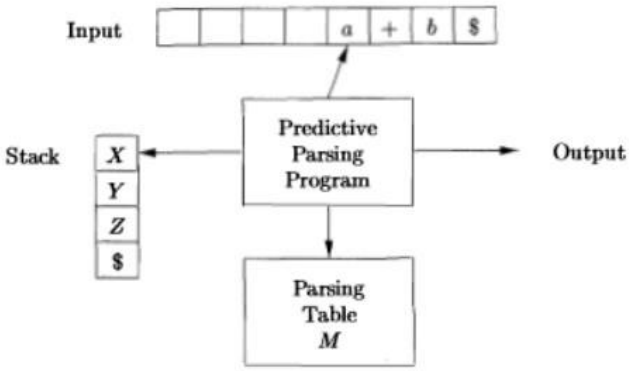
## Non-recursive predictive parsing

**INPUT:** (string w, parsing table M for grammar G)

**OUTPUT:** if w in L(G) produce the leftmost derivation or error (finally I have yo get with empty stack and empty string)

**METHOD:**

set *ip* to point to the first symbol of *w*;
set *X* to the top stack symbol;
**while** ( $X \neq \$$ ) { /* stack is not empty */
    **if** ( *X* is *a* ) pop the stack and advance *ip*;
    **else if** ( *X* is a terminal ) *error*();
    **else if** ( $M[X, a]$ is an error entry ) *error*();
    **else if** ( $M[X, a] = X \rightarrow Y_1 Y_2 \cdots Y_k$ ) {
        output the production $X \rightarrow Y_1 Y_2 \cdots Y_k$;
        pop the stack;
        push $Y_k, Y_{k-1}, \ldots, Y_1$ onto the stack, with $Y_1$ on top;
    }



## Algorithm of Recursive-descent parsing

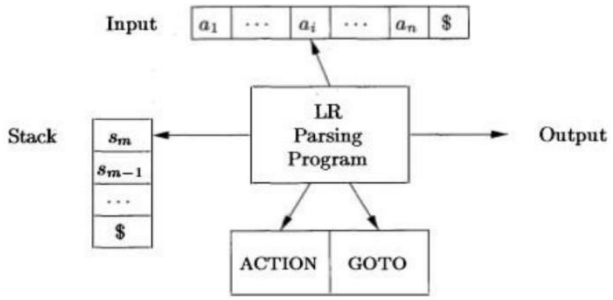
```
void A() {
1)      Choose an A-production, A → X1X2 ··· Xk;
2)      for ( i = 1 to k ) {
3)          if ( Xi is a nonterminal )
4)              call procedure Xi();
5)          else if ( Xi equals the current input symbol a )
6)              advance the input to the next symbol;
7)          else /* an error has occurred */;
        }
    }
```

# Lec 10,11

## Constructing set of items Closure(I)

```
SetOfItems CLOSURE(I) {
    J = I;
    repeat
        for ( each item A → α·Bβ in J )
            for ( each production B → γ of G )
                if ( B → ·γ is not in J )
                    add B → ·γ to J;
    until no more items are added to J on one round;
    return J;
}
```

## LR Parser




## LR-parsing algorithm

**INPUT**: An input string w and an LR-parsing table with functions ACTION and GOTO for a grammar G

**OUTPUT**: If w is in L(G), the reduction steps of a bottom-up parse for w; otherwise, an error indication

**METHOD:**

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
        let s be the state on top of the stack;
        if ( ACTION[s, a] = shift t ) {
                push t onto the stack;
                let a be the next input symbol;
        } else if ( ACTION[s, a] = reduce A → β ) {
                pop |β| symbols off the stack;
                let state t now be on top of the stack;
                push GOTO[t, A] onto the stack;
                output the production A → β;
        } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
        else call error-recovery routine;
}
```

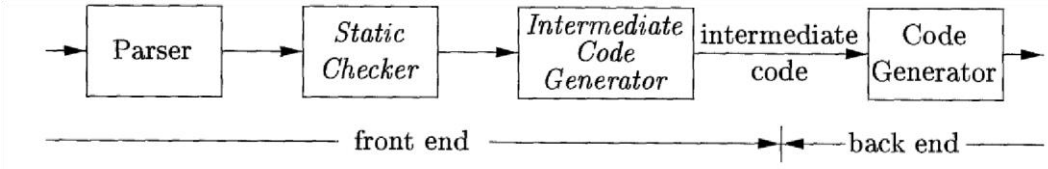
**Syntax-directed translation:** is done by attaching rules or program fragments to productions in a grammar.

**• We can translate expr1 by exploiting its structure, as in the following pseudo-code:**

```
translate expr1;
translate term;
handle +;
```

# Lec 12

## Logical structure of a compiler front end




**A compiler might use a sequence of intermediate representations**




**Three-Address Instructions:** is a sequence of instructions of the form

**Pseudo-code for generating three-address code:**

```
class If extends Stmt {
        Expr E; Stmt S;
        public If(Expr x, Stmt y) { E = x; S = y; after = newlabel(); }
        public void gen() {
                Expr n = E.rvalue();
                emit( "ifFalse " + n.toString() + " goto " + after);
                S.gen();
                emit(after + ":");
        }
}
```

Figure 2.43: Function *gen* in class *If* generates three-address code

## Addresses and Instructions

**A name:** For convenience, we allow source-program names to appear as addresses in three-address code

- **A constant**: In practice, a compiler must deal with many different types of constants and variables **compiler-generated temporary:** It is useful, especially in optimizing compilers

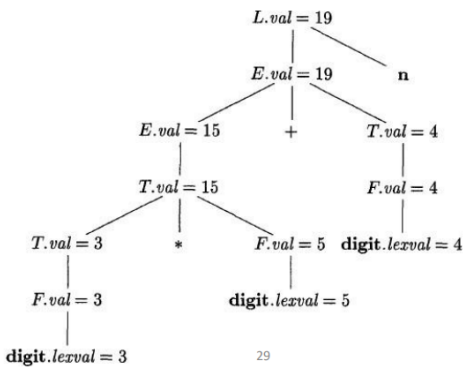
**Directed Acyclic Graph (DAG):** common sub-expressions (sub expressions that occur more than once) of the expression.

**Annotated Parse Tree (APT):** A parse tree showing the values of its attributes is called annotated parse tree.

## Example of the APT

- 3*5+4n (n means end marker)

| | PRODUCTION | SEMANTIC RULES |
|---|---|---|
| 1) | $L \to E$ **n** | $L.val = E.val$ |
| 2) | $E \to E_1 + T$ | $E.val = E_1.val + T.val$ |
| 3) | $E \to T$ | $E.val = T.val$ |
| 4) | $T \to T_1 * F$ | $T.val = T_1.val \times F.val$ |
| 5) | $T \to F$ | $T.val = F.val$ |
| 6) | $F \to (E)$ | $F.val = E.val$ |
| 7) | $F \to$ **digit** | $F.val =$ **digit**.lexval |



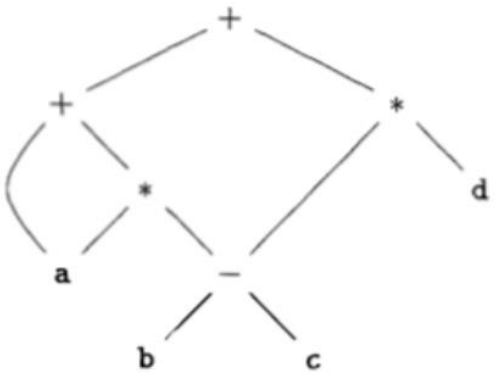
# From SDD to DAG

| | PRODUCTION | SEMANTIC RULES |
|---|---|---|
| 1) | $E \rightarrow E_1 + T$ | $E.node =$ **new** $Node('+', E_1.node, T.node)$ |
| 2) | $E \rightarrow E_1 - T$ | $E.node =$ **new** $Node('-', E_1.node, T.node)$ |
| 3) | $E \rightarrow T$ | $E.node = T.node$ |
| 4) | $T \rightarrow ( E )$ | $T.node = E.node$ |
| 5) | $T \rightarrow$ **id** | $T.node =$ **new** $Leaf($**id**, **id**$.entry)$ |
| 6) | $T \rightarrow$ **num** | $T.node =$ **new** $Leaf($**num**, **num**$.val)$ |

Figure 6.4: Syntax-directed definition to produce syntax trees or DAG's

## Expression: a + a * (b - c) + (b - c) * d

1) $p_1 = Leaf(\textbf{id}, entry\text{-}a)$
2) $p_2 = Leaf(\textbf{id}, entry\text{-}a) = p_1$
3) $p_3 = Leaf(\textbf{id}, entry\text{-}b)$
4) $p_4 = Leaf(\textbf{id}, entry\text{-}c)$
5) $p_5 = Node('-', p_3, p_4)$
6) $p_6 = Node('*', p_1, p_5)$
7) $p_7 = Node('+', p_1, p_6)$
8) $p_8 = Leaf(\textbf{id}, entry\text{-}b) = p_3$
9) $p_9 = Leaf(\textbf{id}, entry\text{-}c) = p_4$
10) $p_{10} = Node('-', p_3, p_4) = p_5$
11) $p_{11} = Leaf(\textbf{id}, entry\text{-}d)$
12) $p_{12} = Node('*', p_5, p_{11})$
13) $p_{13} = Node('+', p_7, p_{12})$




Steps for constructing the DAG of Fig. 6.3